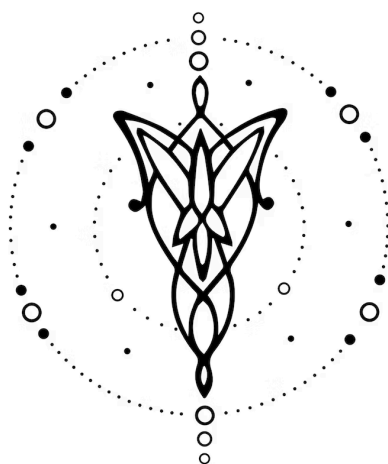




Horarius

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

8 de Marzo de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	4
6. Ejemplos	5

1. Equipo

Nombre	Apellido	Legajo	E-mail
Sofía	Ratto	63.621	sratto@itba.edu.ar
Martina	Nudel	64.155	mnudel@itba.edu.ar
Mateo	Lopez Badias	64.022	mlopezbadias@itba.edu.ar

2. Repositorio

La solución será versionada en: https://github.com/sratto0/TP_TLA

3. Dominio

Se busca desarrollar un lenguaje declarativo que permita describir y validar cronogramas académicos dentro de una institución educativa. El lenguaje estará orientado a modelar los elementos principales del problema: docentes, aulas, materias, comisiones y franjas horarias, junto con sus respectivas asignaciones y restricciones.

La problemática que se busca resolver es la dificultad de organizar horarios de manera consistente cuando intervienen múltiples recursos y restricciones al mismo tiempo. En este contexto, es común que aparezcan conflictos como superposición de docentes, uso simultáneo de un aula, capacidad insuficiente o referencias inválidas entre entidades del sistema.

La implementación de este lenguaje permitiría analizar configuraciones de horarios antes de su aplicación real, detectar conflictos de manera temprana y reducir errores en la planificación académica, mejorando la organización institucional y el uso de recursos.

Para reducir la complejidad del proyecto, el lenguaje se enfocará en la validación de horarios más que en su optimización automática. Es decir, no se buscará generar el “mejor” horario posible, sino verificar si un cronograma dado es válido o inválido según las reglas establecidas.

Todo el procesamiento se realiza dentro del mismo programa, sin interacción con sistemas externos. Las entidades y horarios definidos existen únicamente en memoria, y las verificaciones se aplican sobre estos modelos abstractos.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

(I). Schedule

- (I.1). Se podrá definir un cronograma académico o schedule con un nombre
- (I.2). Cada schedule será independiente de los demás

(II). Docentes

- (II.1). Se podrán declarar uno o más docentes
- (II.2). A cada docente se le podrá asociar disponibilidad horaria
- (II.3). A cada docente se le podrá asociar una o más franjas bloqueadas
- (II.4). Opcionalmente, se podrá especificar una cantidad máxima de asignaciones por día para cada docente

(III). Aulas

- (III.1). Se podrán declarar una o más aulas.
- (III.2). Al declarar un aula, se deberá poder especificar su capacidad.
- (III.3). Opcionalmente, se podrá especificar un tipo de aula.

(IV). Materias y comisiones

- (IV.1). Se podrán declarar una o más materias.

(IV.2). Al declarar una materia, se podrá especificar la cantidad máxima de alumnos.

(IV.3). Se podrán declarar una o más comisiones asociadas a materias previamente definidas.

(V). Franjas horarias

(V.1). Se podrán declarar una o más franjas horarias.

(V.2). Cada franja horaria deberá estar compuesta por un día y un rango horario.

(VI). Asignaciones

(VI.1). Se podrán realizar asignaciones entre una comisión, un docente, un aula y una franja horaria.

(VI.2). Las asignaciones deberán respetar las restricciones del dominio, incluyendo disponibilidad docente, capacidad del aula y no superposición de recursos.

(VII). Validaciones

(VII.1). El lenguaje deberá validar la unicidad de identificadores para entidades del mismo tipo dentro de un mismo schedule.

(VII.2). El lenguaje deberá validar referencias a entidades previamente declaradas.

(VII.3). El lenguaje deberá detectar conflictos de asignación e inconsistencias semánticas.

(VIII). Salida

(VIII.1). Se podrá solicitar la impresión o visualización del cronograma resultante.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Definición de un aula con capacidad mínima y máxima permitida.
- (II). Declaración de un docente con múltiples franjas de disponibilidad horaria.
- (III). Creación de una materia con carga horaria específica y tipo de aula.
- (IV). Asignación válida de una materia a un docente en un aula disponible.
- (V). Asignación de 3 materias diferentes en un mismo aula en franjas horarias contiguas.
- (VI). Declaración de tipo de aula para un aula previamente declarada.
- (VII). Implementación de una distribución de carga horaria entre 2 docentes para una misma materia.
- (VIII). Asignación de un docente a distintas materias sin solapamiento.
- (IX). Creación de un schedule con múltiples materias para un aula determinada, sin solapamiento.
- (X). Asignación de una materia con una capacidad máxima, a un aula específica en una franja horaria sin solapamiento.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Asignación de una materia a un aula que no ha sido definida previamente.
- (II). Asignación de una materia a un aula cuya capacidad es insuficiente para la cantidad de alumnos.
- (III). Asignación de dos materias distintas con misma franja horaria, a un mismo docente.
- (IV). Asignación de una materia a un docente que no ha sido declarado previamente.
- (V). Asignación de dos materias con misma franja horaria, a una misma aula.

6. Ejemplos

En el **Cód. (6.1)**, se ejemplifica un programa válido con dos docentes, dos aulas, y dos materias con sus respectivas comisiones. Luego se establece la disponibilidad de cada docente, y se realizan asignaciones compatibles con los horarios previamente definidos.

```
teacher "Perez";
teacher "Gomez";

room "A101" capacity 40;
room "A102" capacity 30;

course "Automatas" students 35;
course "Compiladores" students 28;

section "Automatas-1" of "Automatas";
section "Compiladores-1" of "Compiladores";

available "Perez" monday 08:00-10:00;
available "Gomez" monday 10:00-12:00;

assign "Automatas-1" to "Perez" in "A101" at monday 08:00-10:00;
assign "Compiladores-1" to "Gomez" in "A102" at monday 10:00-12:00;

print schedule;
```

Código 6.1: ejemplo de programa válido.

En el **Cód. (6.2)**, se muestra un ejemplo de programa inválido. Se intenta asignar a un mismo docente a dos comisiones distintas dentro de la misma franja horaria. Esto viola una restricción semántica del dominio, y por ende el programa es inválido.

```
teacher "Perez";

room "A101" capacity 40;
room "A102" capacity 50;

course "Automatas" students 35;
course "Compiladores" students 20;

section "Automatas-1" of "Automatas";
section "Compiladores-1" of "Compiladores";

available "Perez" monday 08:00-10:00;

assign "Automatas-1" to "Perez" in "A101" at monday 08:00-10:00;
assign "Compiladores-1" to "Perez" in "A102" at monday 08:00-10:00;
```

Código 6.2: ejemplo de programa inválido.